PROYECTO FINAL IA 2

DIEGO ANDRES LOZADA NIÑO 2170078

LEWING ANDRES MENDEZ ORTIZ 2162120

RETO

Data Explorer

343.51 GB

- segmentations
- test_images
- train_images
- sample_submission.csv
- test.csv
- train.csv
- train_bounding_boxes.csv

https://www.kaggle.com/competitions/rsna-2022-cervical-spine-fracture-detection/data

TRATAMIENTO DE DATOS

- El dataset incluye 2019 pacientes, de los cuales se utilizaran 655

Se dividieron las imagenes en 7, ya que en la informacion dada por la organizacion que recopilo la informacion se indica que se poseen la tomografia completa de toda la espina cervical. por ello se dividen la cantidad de imagenes en 7.

```
[ ] print('paciente: 1.2.826.0.1.3680043.10014' )
    print('densidad tomografia: ', len(os.listdir(PATH + '/rsna/train_images/1.2.826.0.1.3680043.10014/')))
    print('densidad de tomografias por vertebra: ', int(len(os.listdir(PATH + '/rsna/train_images/1.2.826.0.1.3680043.10014/'))/7))
    print('img inicio vertebra rota: ', len(os.listdir(PATH + '/rsna/train_images/1.2.826.0.1.3680043.10014/'))-64)
```

```
paciente: 1.2.826.0.1.3680043.10014
densidad tomografia:  258
densidad de tomografias por vertebra:  36
img inicio vertebra rota:  194
```

	StudyInstanceUID	patient_overall	C1	C2	C3	C4	C5	C6	C7
353	1.2.826.0.1.3680043.30020	1	0	0	0	0	1	0	0
896	1.2.826.0.1.3680043.30030	1	0	0	0	0	0	1	1
1162	1.2.826.0.1.3680043.30037	0	0	0	0	0	0	0	0
1905	1.2.826.0.1.3680043.30038	0	0	0	0	0	0	0	0
947	1.2.826.0.1.3680043.30039	1	0	0	0	0	0	0	1

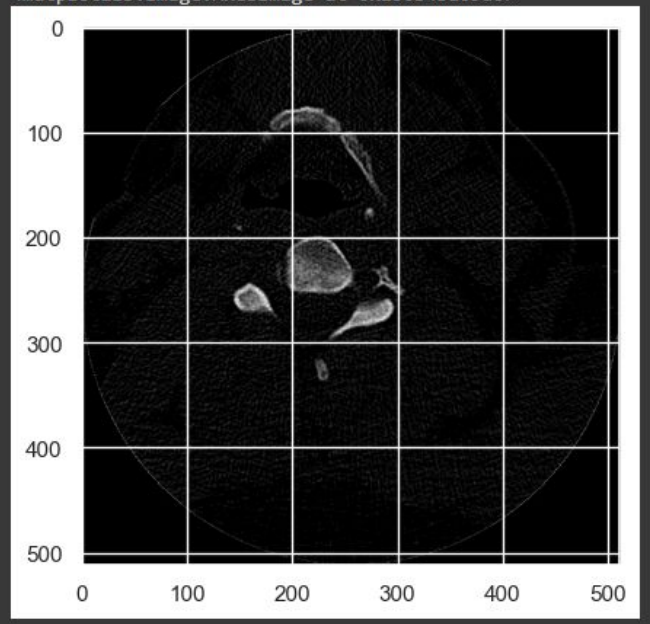
INFORMACIÓN DETALLADA IMÁGENES DICOM

```
[ ] print(dicom)

    Dataset.file_meta -------------------------------
    (0002, 0001) File Meta Information Version       OB: b'\x00\x01'
    (0002, 0002) Media Storage SOP Class UID         UI: CT Image Storage
    (0002, 0003) Media Storage SOP Instance UID      UI: 1.2.826.0.1.3680043.1010.1.1
    (0002, 0010) Transfer Syntax UID                 UI: JPEG Lossless, Non-Hierarchical, First-Order Prediction (Process 14 [Selection Value 1])
    (0002, 0012) Implementation Class UID            UI: 1.2.40.0.13.1.1.1
    (0002, 0013) Implementation Version Name         SH: 'PYDICOM 2.3.0'
    -------------------------------------------------
    (0008, 0018) SOP Instance UID                    UI: 1.2.826.0.1.3680043.1010.1.1
    (0008, 0023) Content Date                        DA: '20220727'
    (0008, 0033) Content Time                        TM: '185141.910658'
    (0010, 0010) Patient's Name                      PN: '1010'
    (0010, 0020) Patient ID                          LO: '1010'
    (0018, 0050) Slice Thickness                     DS: '1.0'
    (0020, 000d) Study Instance UID                  UI: 1.2.826.0.1.3680043.1010
    (0020, 000e) Series Instance UID                 UI: 1.2.826.0.1.3680043.1010.1
    (0020, 0013) Instance Number                     IS: '1'
    (0020, 0032) Image Position (Patient)            DS: [-105.461, -86.71143, -456.00]
    (0020, 0037) Image Orientation (Patient)         DS: [1.00000, 0.00000, 0.00000, 0.00000, 1.00000, 0.00000]
    (0028, 0002) Samples per Pixel                   US: 1
    (0028, 0004) Photometric Interpretation          CS: 'MONOCHROME2'
    (0028, 0010) Rows                                US: 512
    (0028, 0011) Columns                             US: 512
    (0028, 0030) Pixel Spacing                       DS: [0.327, 0.327]
    (0028, 0100) Bits Allocated                      US: 16
    (0028, 0101) Bits Stored                         US: 16
    (0028, 0102) High Bit                            US: 15
    (0028, 0103) Pixel Representation                US: 1
    (0028, 1050) Window Center                       DS: '350.0'
    (0028, 1051) Window Width                        DS: '2700.0'
    (0028, 1052) Rescale Intercept                   DS: '0.0'
    (0028, 1053) Rescale Slope                       DS: '1.0'
    (7fe0, 0010) Pixel Data                          OB: Array of 266850 elements
```

```
[ ] img_convertida = cv2.cvtColor(pixel_arrays[20], cv2.COLOR_GRAY2RGB)
    plt.imshow(img_convertida)

    <matplotlib.image.AxesImage at 0x250b4ba09d0>
```



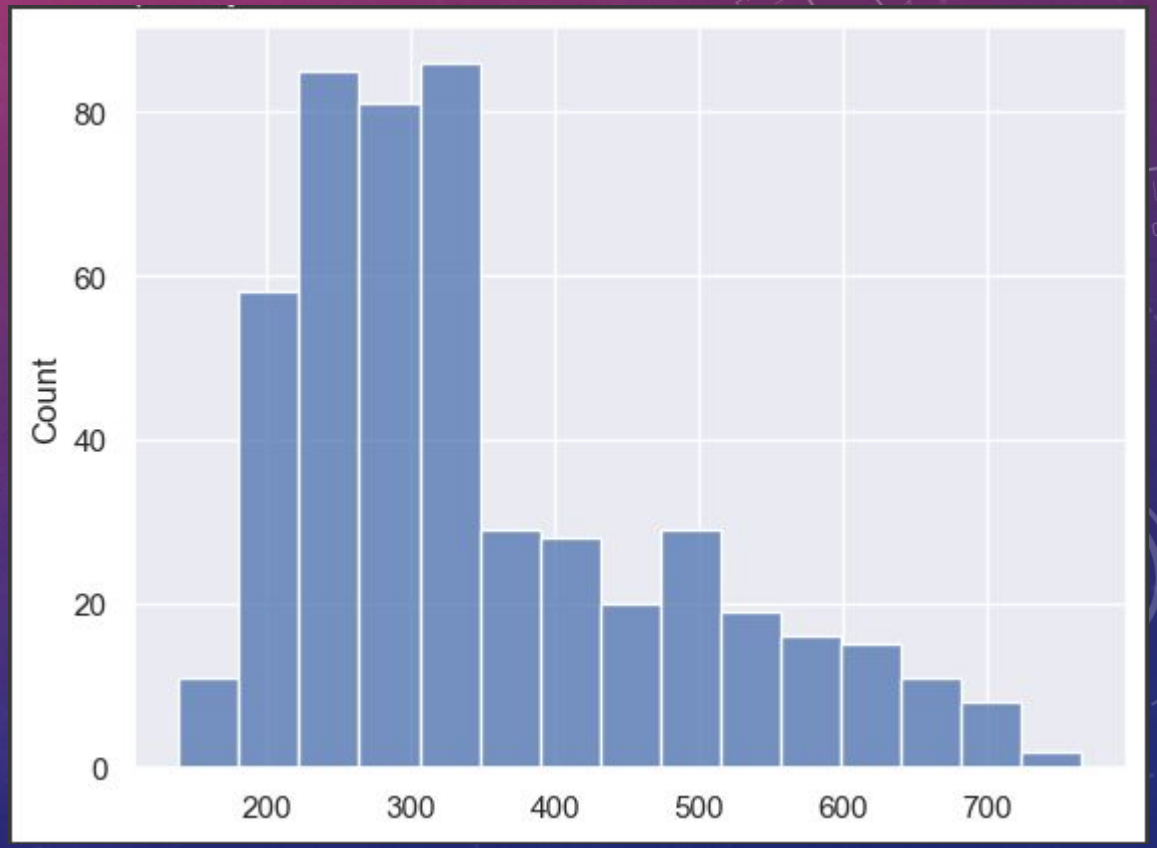
CANTIDAD DE IMÁGENES POR PACIENTE

cantidad de imagenes por dimension, para normalizar la data

```
[ ] cantidad = []
    for i in l_labels:
      cantidad.append(len(os.listdir(PATH + 'rsna/train_images/' + i)))
```

```
[ ] print(max(cantidad))
    print(min(cantidad))
    print(np.mean(cantidad))
```

```
765
139
347.9859437751004
```



En esta parte se almacenan las imagenes por paciente en la carpeta respectiva a la vertebra a la que pertence la imagen, esto en base al numero de imagen y la cantidad de imagenes, por esto mismo se revisa que el nombre de la imagen coincida con la cantidad de imagenes, es decir si tiene 300 imagenes se espera que el ultimo archivo dicom se llame 300.dcm.

```
###no borrar, transforma los .dcm en jpg blanco y negro

pacientes = l_labels
pyd = []
for p in pacientes:
  paciente = sorted(os.listdir(PATH + '/rsna/test_images/' + p))
  print('paciente: ', p)
  print('len list: ' + str(len(paciente)))

  #Creacion de carpetas por pacietne y sus vertebras
  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p)

  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/c1')
  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/c2')
  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/c3')
  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/c4')
  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/c5')
  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/c6')
  os.mkdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/c7')

  tamaño_vertebra = int(len(os.listdir(PATH + '/rsna/test_images/' + p))/7)
  print('cantidad de imgs por vertebra: ', tamaño_vertebra)
```

```
##Calcula el # maximo de la vertebra
valor_max = 0
for i in paciente:
  if (int(i[:-4])>valor_max):
    valor_max = int(i[:-4])

v_sum = abs(len(paciente)-valor_max)

print({'cantidad-imgs: ':len(paciente), 'ultima-img: ':valor_max, 'diferencia: ':v_sum})
```

```python
for i in paciente:
  #Transformacion de la imagen
  img = pydicom.dcmread(PATH + 'rsna/test_images/' + p + '/' + i).pixel_array
  img_scaled = np.uint8((np.maximum(img, 0)/img.max())*255.0)

  #Se guarda la imagen segun su ubicacion en vertebra
  num_img = int(i[:-4]) - v_sum

  if (num_img <= tamaño_vertebra):
    cv2.imwrite(PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/C1/' + i[:-4] +'.jpg' , img_scaled)

  if (tamaño_vertebra < num_img <= (tamaño_vertebra*2)):
    cv2.imwrite(PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/C2/' + i[:-4] +'.jpg' , img_scaled)

  if (tamaño_vertebra*2 < num_img <= tamaño_vertebra*3):
    cv2.imwrite(PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/C3/' + i[:-4] +'.jpg' , img_scaled)

  if (tamaño_vertebra*3 < num_img <= tamaño_vertebra*4):
    cv2.imwrite(PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/C4/' + i[:-4] +'.jpg' , img_scaled)

  if (tamaño_vertebra*4 < num_img <= tamaño_vertebra*5):
    cv2.imwrite(PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/C5/' + i[:-4] +'.jpg' , img_scaled)

  if (tamaño_vertebra*5 < num_img <= tamaño_vertebra*6):
    cv2.imwrite(PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/C6/' + i[:-4] +'.jpg' , img_scaled)

  if (tamaño_vertebra*6 < num_img):
    cv2.imwrite(PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/C7/' + i[:-4] +'.jpg' , img_scaled)

print(p)
path = PATH + 'rsna/imgs_por_vertebra_test/'+ p + '/'
s = (len(os.listdir(path+'C1'))+len(os.listdir(path+'C2'))+len(os.listdir(path+'C3'))+
    len(os.listdir(path+'C4'))+len(os.listdir(path+'C5'))+len(os.listdir(path+'C6'))+len(os.listdir(path+'C7')))
print('len dir: ', s)
```

Se crean las carpetas para la clasificacion de imagenes por vertebras, ya que al poseer tantas imagenes se deben cargar desde el disco duro, debido a que se usara la libreria tf.keras.preprocessing.image_dataset_from_directory se debe guardar las imagenes por clase, para esto se define que existen 14 clases donde 7 hacen referencia a la vertebra sin fractura, las carpetas terminadas en 0, y otras 7 carpetas que hacen referencia a las 7 vertebras con fractura.

```
[ ] #os.mkdir(PATH + 'rsna/imgs_vertebra_estado/' + p)

    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C11')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C10')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C21')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C20')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C31')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C30')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C41')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C40')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C51')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C50')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C61')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C60')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C71')
    os.mkdir(PATH + 'rsna/imgs_vertebra_estado_test/C70')
```

Posteriormente se copian las imagenes por paciente a su respectiva carpeta de clase, con un nombre que indica que paciente y numero de imagen posee, para esto se define, asignar en las clases con ruptura de vertebra solo las imagenes de cuyo paciente poseia fractura en el rango especifico, posterior a esto se asignan de forma semejante en las clases sin fractura las imagenes de las vertebras cuyo paciente no tuvo ruptura en alguna vertebra, por esto mismo se espera obtener mas imagenes en las clases sin fractura cervical, por lo que posteriormente se realizara una normalizacion para que tanto las clases con fractura y sin fractura posean numero similares de imagenes, esto con el objetivo de no sesgar el aprendizaje.

```
[ ] #shutil.copy("archivo.txt", "Documentos/archivo.txt")
    #data = labels.iloc[1]
    print(data.shape[0])

    vertebras = ['C1', 'C2', 'C3', 'C4', 'C5', 'C6', 'C7']

    for i in range(0, data.shape[0]):
      print(data['StudyInstanceUID'].iloc[i])
      for vertebra in vertebras:
        p = data['StudyInstanceUID'].iloc[i]

        if ((data[vertebra].iloc[i] == 1) & (data['patient_overall'].iloc[i] == 1)):
          for img in os.listdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/' + vertebra):
            f = PATH + 'rsna/imgs_por_vertebra_test/' + p + '/' + vertebra + '/' + img
            to = PATH + 'rsna/imgs_vertebra_estado_test/' + vertebra + '1' + '/'+ p + '.' + img
            shutil.copy(f, to)

        if ( data['patient_overall'].iloc[i] == 0 ):
          for img in os.listdir(PATH + 'rsna/imgs_por_vertebra_test/' + p + '/' + vertebra):
            f = PATH + 'rsna/imgs_por_vertebra_test/' + p + '/' + vertebra + '/' + img
            to = PATH + 'rsna/imgs_vertebra_estado_test/' + vertebra + '0' + '/'+ p + '.' + img
            shutil.copy(f, to)
```

```
[ ] #vertebras = ['C1', 'C2', 'C3', 'C4', 'C5', 'C6', 'C7']

    counts = []
    counts_sin = 0

    vertebras = os.listdir(PATH + 'rsna/imgs_vertebra_estado_test/')
    print(vertebras)

    for vertebra in vertebras:
      cant_img = len(os.listdir(PATH + 'rsna/imgs_vertebra_estado_test/' + vertebra))
      counts.append(cant_img)
      print('La vertebra ',vertebra, ' tiene ', cant_img, ' imagenes sin ruptura.')

    print('total tomografias con fractura: ', np.sum(counts))
```

```
['C10', 'C11', 'C20', 'C21', 'C30', 'C31', 'C40', 'C41', 'C50', 'C51', 'C60', 'C61', 'C70', 'C71']
La vertebra  C10  tiene  2122  imagenes sin ruptura.
La vertebra  C11  tiene  337  imagenes sin ruptura.
La vertebra  C20  tiene  2122  imagenes sin ruptura.
La vertebra  C21  tiene  234  imagenes sin ruptura.
La vertebra  C30  tiene  2122  imagenes sin ruptura.
La vertebra  C31  tiene  162  imagenes sin ruptura.
La vertebra  C40  tiene  2122  imagenes sin ruptura.
La vertebra  C41  tiene  172  imagenes sin ruptura.
La vertebra  C50  tiene  2122  imagenes sin ruptura.
La vertebra  C51  tiene  320  imagenes sin ruptura.
La vertebra  C60  tiene  2122  imagenes sin ruptura.
La vertebra  C61  tiene  486  imagenes sin ruptura.
La vertebra  C70  tiene  2244  imagenes sin ruptura.
La vertebra  C71  tiene  920  imagenes sin ruptura.
total tomografias con fractura:  17607
```

```python
counts_con = []
counts_sin = []

vertebras = os.listdir(PATH + 'rsna/imgs_vertebra_estado_norm/')
print(vertebras)
con = 0
for vertebra in vertebras:
  con = con +1
  cant_img = len(os.listdir(PATH + 'rsna/imgs_vertebra_estado_norm/' + vertebra))
  if(con%2 == 0):
    counts_con.append(cant_img)
  else:
    counts_sin.append(cant_img)


  counts.append(cant_img)
  print('La vertebra ',vertebra, ' tiene ', cant_img, ' imagenes sin ruptura.')

print('total tomografias con fractura: ', np.sum(counts_con))
print('total tomografias sin fractura: ', np.sum(counts_sin))
```

```
['C10', 'C11', 'C20', 'C21', 'C30', 'C31', 'C40', 'C41', 'C50', 'C51', 'C60', 'C61', 'C70', 'C71']
La vertebra  C10  tiene  5532  imagenes sin ruptura.
La vertebra  C11  tiene  3966  imagenes sin ruptura.
La vertebra  C20  tiene  5532  imagenes sin ruptura.
La vertebra  C21  tiene  6599  imagenes sin ruptura.
La vertebra  C30  tiene  5532  imagenes sin ruptura.
La vertebra  C31  tiene  2147  imagenes sin ruptura.
La vertebra  C40  tiene  5532  imagenes sin ruptura.
La vertebra  C41  tiene  3360  imagenes sin ruptura.
La vertebra  C50  tiene  5532  imagenes sin ruptura.
La vertebra  C51  tiene  4740  imagenes sin ruptura.
La vertebra  C60  tiene  5532  imagenes sin ruptura.
La vertebra  C61  tiene  7871  imagenes sin ruptura.
La vertebra  C70  tiene  5532  imagenes sin ruptura.
La vertebra  C71  tiene  10047  imagenes sin ruptura.
total tomografias con fractura:  38730
total tomografias sin fractura:  38724
```

PROCESAMIENTO DE IMÁGENES

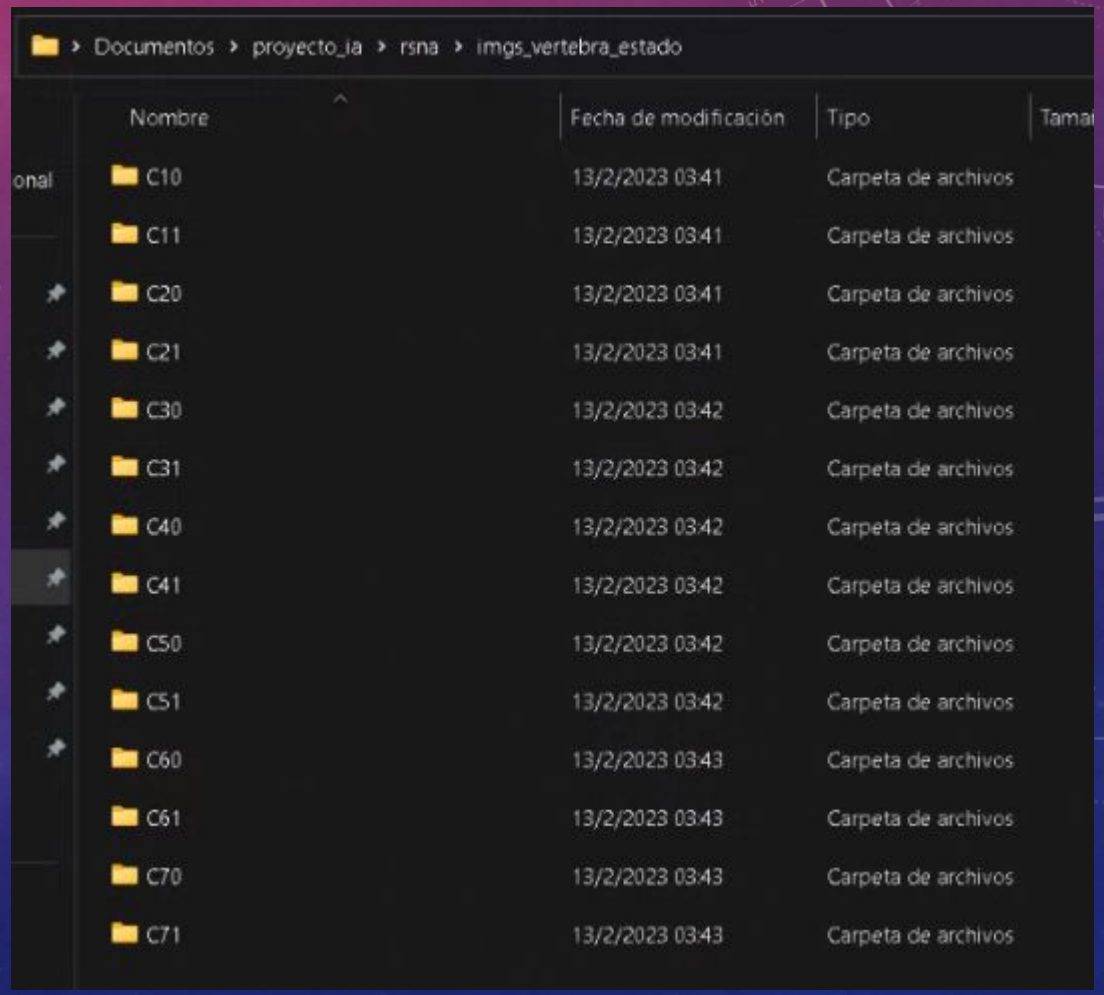
- Se reescalan las imágenes a 384*384 un solo canal

```python
#train_ds = tf.keras.preprocessing.image_dataset_from_directory(
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
  PATH + 'imgs_vertebra_estado',
  validation_split=0.2,
  subset="training",
  seed=123,
  label_mode='categorical',
  color_mode = "grayscale",
  image_size=(img_height, img_width),
  batch_size=batch_size)
```

```
Found 134907 files belonging to 14 classes.
Using 107926 files for training.
```

```python
#val_ds = tf.keras.preprocessing.image_dataset_from_directory(
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
  PATH + 'imgs_vertebra_estado',
  validation_split=0.2,
  subset="validation",
  seed=123,
  label_mode='categorical',
  color_mode = "grayscale",
  image_size=(img_height, img_width),
  batch_size=batch_size)
```

```
Found 134907 files belonging to 14 classes.
Using 26981 files for validation.
```



DISTRIBUCIÓN DE DATOS SIN NORMALIZAR

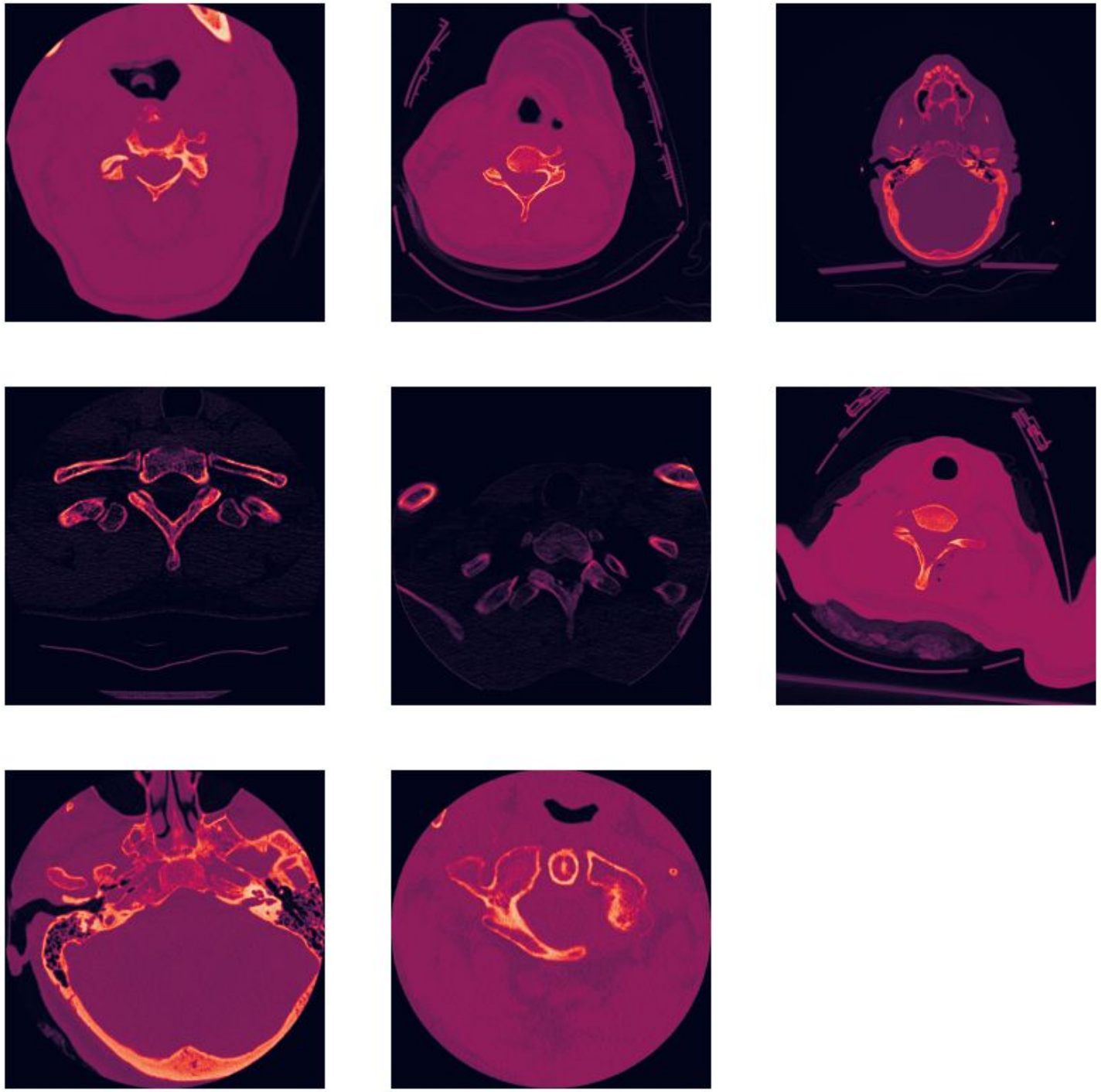
```
['C10', 'C11', 'C20', 'C21', 'C30', 'C31', 'C40', 'C41', 'C50', 'C51', 'C60', 'C61', 'C70', 'C71']
La vertebra  C10  tiene  13877  imagenes sin ruptura.
La vertebra  C11  tiene  3629  imagenes sin ruptura.
La vertebra  C20  tiene  13877  imagenes sin ruptura.
La vertebra  C21  tiene  6365  imagenes sin ruptura.
La vertebra  C30  tiene  13877  imagenes sin ruptura.
La vertebra  C31  tiene  1985  imagenes sin ruptura.
La vertebra  C40  tiene  13877  imagenes sin ruptura.
La vertebra  C41  tiene  3188  imagenes sin ruptura.
La vertebra  C50  tiene  13877  imagenes sin ruptura.
La vertebra  C51  tiene  4420  imagenes sin ruptura.
La vertebra  C60  tiene  13877  imagenes sin ruptura.
La vertebra  C61  tiene  7385  imagenes sin ruptura.
La vertebra  C70  tiene  14626  imagenes sin ruptura.
La vertebra  C71  tiene  10047  imagenes sin ruptura.
total tomografias con fractura:  134907
```

DISTRIBUCIÓN DE DATOS NORMALIZADOS

```
['C10', 'C11', 'C20', 'C21', 'C30', 'C31', 'C40', 'C41', 'C50', 'C51', 'C60', 'C61', 'C70', 'C71']
La vertebra  C10  tiene  5532  imagenes sin ruptura.
La vertebra  C11  tiene  3966  imagenes sin ruptura.
La vertebra  C20  tiene  5532  imagenes sin ruptura.
La vertebra  C21  tiene  6599  imagenes sin ruptura.
La vertebra  C30  tiene  5532  imagenes sin ruptura.
La vertebra  C31  tiene  2147  imagenes sin ruptura.
La vertebra  C40  tiene  5532  imagenes sin ruptura.
La vertebra  C41  tiene  3360  imagenes sin ruptura.
La vertebra  C50  tiene  5532  imagenes sin ruptura.
La vertebra  C51  tiene  4740  imagenes sin ruptura.
La vertebra  C60  tiene  5532  imagenes sin ruptura.
La vertebra  C61  tiene  7871  imagenes sin ruptura.
La vertebra  C70  tiene  5532  imagenes sin ruptura.
La vertebra  C71  tiene  10047  imagenes sin ruptura.
total tomografias con fractura:  38730
total tomografias sin fractura:  38724
```

VISUALIZANDO, BATCH DE ENTRENAMIENTO CON IMAGEN Y SU ETIQUETA

```
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]],
```



```
model =  tf.keras.models.Sequential()
model.add(tf.keras.layers.Conv2D(16, (3, 3), activation='relu', padding='same', input_shape=(384, 384, 1)))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Conv2D(16, (3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.MaxPooling2D((2, 2), padding='same'))
model.add(tf.keras.layers.Dropout(0.3))

model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.MaxPooling2D((2, 2), padding='same'))
model.add(tf.keras.layers.Dropout(0.3))

model.add(tf.keras.layers.Conv2D(48, (3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Conv2D(48, (3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.MaxPooling2D((2, 2), padding='same'))
model.add(tf.keras.layers.Dropout(0.3))

model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(tf.keras.layers.BatchNormalization())
model.add(tf.keras.layers.MaxPooling2D((4, 4), padding='same'))
model.add(tf.keras.layers.Dropout(0.5))


model.add(tf.keras.layers.Flatten())

model.add(tf.keras.layers.Dense(14, activation='relu'))
model.add(tf.keras.layers.Dense(14, activation='softmax'))
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_32 (Conv2D)	(None, 384, 384, 16)	160
batch_normalization_32 (Batc	(None, 384, 384, 16)	64
conv2d_33 (Conv2D)	(None, 384, 384, 16)	2320
batch_normalization_33 (Batc	(None, 384, 384, 16)	64
max_pooling2d_16 (MaxPooling	(None, 192, 192, 16)	0
dropout_10 (Dropout)	(None, 192, 192, 16)	0
conv2d_34 (Conv2D)	(None, 192, 192, 32)	4640
batch_normalization_34 (Batc	(None, 192, 192, 32)	128
conv2d_35 (Conv2D)	(None, 192, 192, 32)	9248
batch_normalization_35 (Batc	(None, 192, 192, 32)	128
max_pooling2d_17 (MaxPooling	(None, 96, 96, 32)	0
dropout_11 (Dropout)	(None, 96, 96, 32)	0
conv2d_36 (Conv2D)	(None, 96, 96, 48)	13872
batch_normalization_36 (Batc	(None, 96, 96, 48)	192
conv2d_37 (Conv2D)	(None, 96, 96, 48)	20784
batch_normalization_37 (Batc	(None, 96, 96, 48)	192
max_pooling2d_18 (MaxPooling	(None, 48, 48, 48)	0
dropout_12 (Dropout)	(None, 48, 48, 48)	0
conv2d_38 (Conv2D)	(None, 48, 48, 64)	27712
batch_normalization_38 (Batc	(None, 48, 48, 64)	256
conv2d_39 (Conv2D)	(None, 48, 48, 64)	36928
batch_normalization_39 (Batc	(None, 48, 48, 64)	256
max_pooling2d_19 (MaxPooling	(None, 12, 12, 64)	0
dropout_13 (Dropout)	(None, 12, 12, 64)	0
flatten_4 (Flatten)	(None, 9216)	0
dense_8 (Dense)	(None, 14)	129038
dense_9 (Dense)	(None, 14)	210

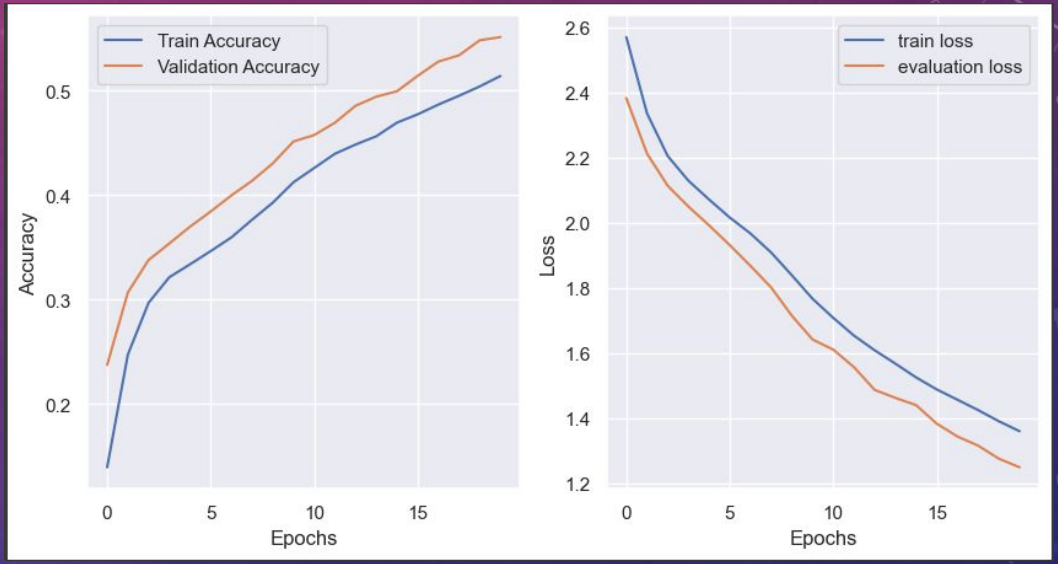
ENTRENAMIENTO

```
opt = tf.keras.optimizers.Adadelta(learning_rate=0.01)
model.compile(optimizer=opt, loss='categorical_crossentropy',metrics=['accuracy'])
```

- Para el primer entrenamiento se utilizaron 20 epochs.
- Para el resto de entrenamientos se utilizaron 10
- El mejor optimizador para el entrenamiento de manera empírica fue adadelta.

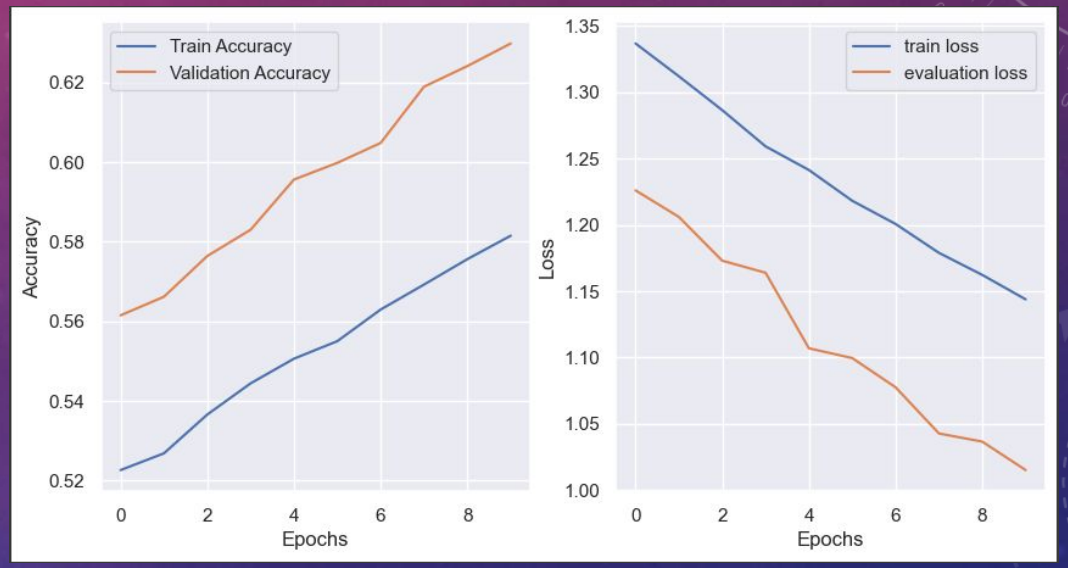
PRIMER ENTRENAMIENTO

```
Epoch 1/20
13491/13491 [==============================] - 1690s 125ms/step - loss: 2.5701 - accuracy: 0.1397 - val_loss: 2.3827 - val_accuracy: 0.2376
Epoch 2/20
13491/13491 [==============================] - 1691s 125ms/step - loss: 2.3360 - accuracy: 0.2475 - val_loss: 2.2125 - val_accuracy: 0.3070
Epoch 3/20
13491/13491 [==============================] - 1692s 125ms/step - loss: 2.2049 - accuracy: 0.2971 - val_loss: 2.1136 - val_accuracy: 0.3381
Epoch 4/20
13491/13491 [==============================] - 1693s 126ms/step - loss: 2.1292 - accuracy: 0.3215 - val_loss: 2.0497 - val_accuracy: 0.3538
Epoch 5/20
13491/13491 [==============================] - 1684s 125ms/step - loss: 2.0713 - accuracy: 0.3339 - val_loss: 1.9919 - val_accuracy: 0.3699
Epoch 6/20
13491/13491 [==============================] - 1686s 125ms/step - loss: 2.0159 - accuracy: 0.3466 - val_loss: 1.9312 - val_accuracy: 0.3845
Epoch 7/20
13491/13491 [==============================] - 1686s 125ms/step - loss: 1.9669 - accuracy: 0.3596 - val_loss: 1.8675 - val_accuracy: 0.3999
Epoch 8/20
13491/13491 [==============================] - 1684s 125ms/step - loss: 1.9084 - accuracy: 0.3766 - val_loss: 1.8011 - val_accuracy: 0.4138
Epoch 9/20
13491/13491 [==============================] - 1683s 125ms/step - loss: 1.8384 - accuracy: 0.3929 - val_loss: 1.7143 - val_accuracy: 0.4305
Epoch 10/20
13491/13491 [==============================] - 1685s 125ms/step - loss: 1.7663 - accuracy: 0.4124 - val_loss: 1.6413 - val_accuracy: 0.4515
Epoch 11/20
13491/13491 [==============================] - 1687s 125ms/step - loss: 1.7078 - accuracy: 0.4261 - val_loss: 1.6103 - val_accuracy: 0.4576
Epoch 12/20
13491/13491 [==============================] - 1689s 125ms/step - loss: 1.6533 - accuracy: 0.4397 - val_loss: 1.5564 - val_accuracy: 0.4694
Epoch 13/20
13491/13491 [==============================] - 1689s 125ms/step - loss: 1.6083 - accuracy: 0.4486 - val_loss: 1.4866 - val_accuracy: 0.4856
Epoch 14/20
13491/13491 [==============================] - 1690s 125ms/step - loss: 1.5672 - accuracy: 0.4565 - val_loss: 1.4621 - val_accuracy: 0.4943
Epoch 15/20
13491/13491 [==============================] - 1692s 125ms/step - loss: 1.5249 - accuracy: 0.4695 - val_loss: 1.4398 - val_accuracy: 0.4995
Epoch 16/20
13491/13491 [==============================] - 1691s 125ms/step - loss: 1.4880 - accuracy: 0.4775 - val_loss: 1.3828 - val_accuracy: 0.5143
Epoch 17/20
13491/13491 [==============================] - 1692s 125ms/step - loss: 1.4564 - accuracy: 0.4868 - val_loss: 1.3434 - val_accuracy: 0.5280
Epoch 18/20
13491/13491 [==============================] - 1691s 125ms/step - loss: 1.4244 - accuracy: 0.4952 - val_loss: 1.3154 - val_accuracy: 0.5340
Epoch 19/20
13491/13491 [==============================] - 1677s 124ms/step - loss: 1.3903 - accuracy: 0.5042 - val_loss: 1.2753 - val_accuracy: 0.5483
Epoch 20/20
13491/13491 [==============================] - 1669s 124ms/step - loss: 1.3596 - accuracy: 0.5142 - val_loss: 1.2490 - val_accuracy: 0.5514
```



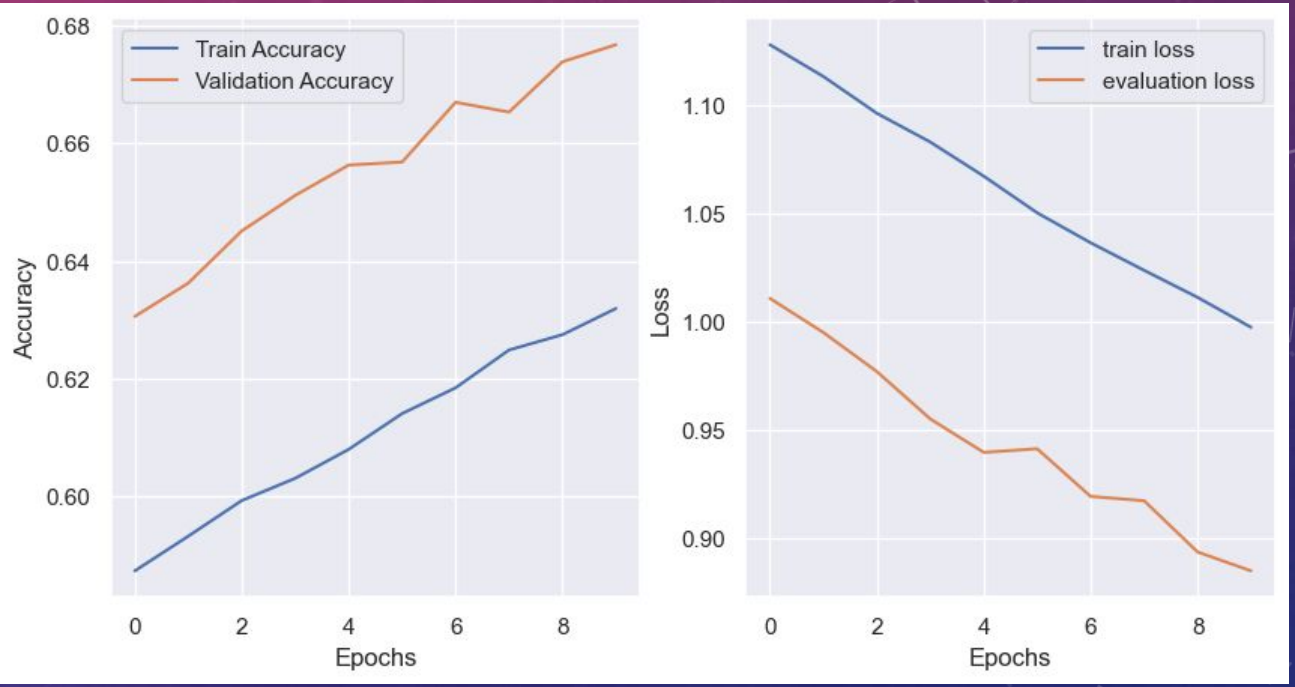
SEGUNDO ENTRENAMIENTO

```
Epoch 1/10
    2/13491 [..............................] - ETA: 30:23 - loss: 1.3447 - accuracy: 0.4375WARNING:tensorflow:Callbacks method `on_train_batc
13491/13491 [==============================] - 1707s 127ms/step - loss: 1.3368 - accuracy: 0.5226 - val_loss: 1.2261 - val_accuracy: 0.5615
Epoch 2/10
13491/13491 [==============================] - 1690s 125ms/step - loss: 1.3121 - accuracy: 0.5269 - val_loss: 1.2060 - val_accuracy: 0.5662
Epoch 3/10
13491/13491 [==============================] - 1681s 125ms/step - loss: 1.2866 - accuracy: 0.5366 - val_loss: 1.1731 - val_accuracy: 0.5764
Epoch 4/10
13491/13491 [==============================] - 1682s 125ms/step - loss: 1.2592 - accuracy: 0.5444 - val_loss: 1.1640 - val_accuracy: 0.5830
Epoch 5/10
13491/13491 [==============================] - 1684s 125ms/step - loss: 1.2414 - accuracy: 0.5506 - val_loss: 1.1070 - val_accuracy: 0.5956
Epoch 6/10
13491/13491 [==============================] - 1690s 125ms/step - loss: 1.2182 - accuracy: 0.5550 - val_loss: 1.0996 - val_accuracy: 0.5998
Epoch 7/10
13491/13491 [==============================] - 1685s 125ms/step - loss: 1.2007 - accuracy: 0.5630 - val_loss: 1.0776 - val_accuracy: 0.6048
Epoch 8/10
13491/13491 [==============================] - 1666s 123ms/step - loss: 1.1789 - accuracy: 0.5692 - val_loss: 1.0428 - val_accuracy: 0.6189
Epoch 9/10
13491/13491 [==============================] - 1661s 123ms/step - loss: 1.1623 - accuracy: 0.5756 - val_loss: 1.0366 - val_accuracy: 0.6241
Epoch 10/10
13491/13491 [==============================] - 1692s 125ms/step - loss: 1.1439 - accuracy: 0.5815 - val_loss: 1.0152 - val_accuracy: 0.6298
```



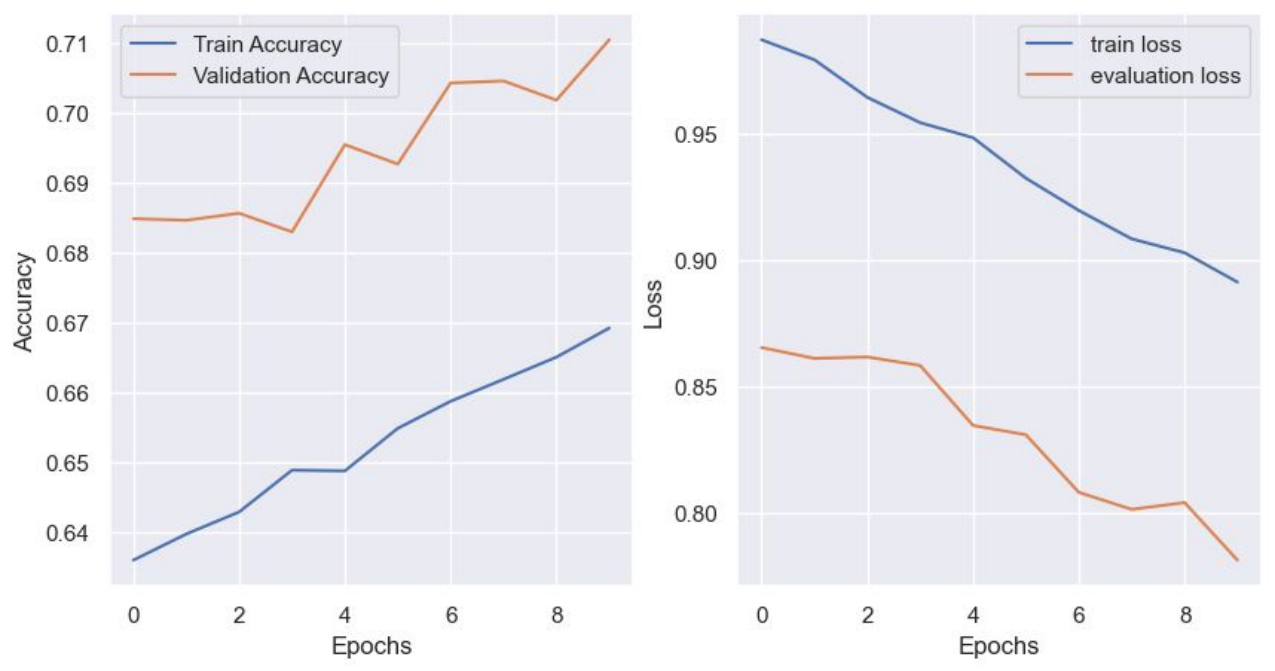
TERCER ENTRENAMIENTO

```
Epoch 1/10
    2/13491 [..............................] - ETA: 29:01 - loss: 1.3874 - accuracy: 0.3750WARNING:tensorflow:Callbacks method `on_train_batc
13491/13491 [==============================] - 1673s 124ms/step - loss: 1.1276 - accuracy: 0.5873 - val_loss: 1.0108 - val_accuracy: 0.6306
Epoch 2/10
13491/13491 [==============================] - 1672s 124ms/step - loss: 1.1130 - accuracy: 0.5932 - val_loss: 0.9950 - val_accuracy: 0.6362
Epoch 3/10
13491/13491 [==============================] - 1666s 123ms/step - loss: 1.0960 - accuracy: 0.5993 - val_loss: 0.9770 - val_accuracy: 0.6451
Epoch 4/10
13491/13491 [==============================] - 1668s 124ms/step - loss: 1.0828 - accuracy: 0.6030 - val_loss: 0.9552 - val_accuracy: 0.6511
Epoch 5/10
13491/13491 [==============================] - 1666s 124ms/step - loss: 1.0671 - accuracy: 0.6080 - val_loss: 0.9398 - val_accuracy: 0.6563
Epoch 6/10
13491/13491 [==============================] - 1667s 124ms/step - loss: 1.0502 - accuracy: 0.6140 - val_loss: 0.9415 - val_accuracy: 0.6568
Epoch 7/10
13491/13491 [==============================] - 1668s 124ms/step - loss: 1.0364 - accuracy: 0.6184 - val_loss: 0.9195 - val_accuracy: 0.6670
Epoch 8/10
13491/13491 [==============================] - 1671s 124ms/step - loss: 1.0237 - accuracy: 0.6248 - val_loss: 0.9175 - val_accuracy: 0.6653
Epoch 9/10
13491/13491 [==============================] - 1672s 124ms/step - loss: 1.0112 - accuracy: 0.6274 - val_loss: 0.8939 - val_accuracy: 0.6738
Epoch 10/10
13491/13491 [==============================] - 1682s 125ms/step - loss: 0.9974 - accuracy: 0.6319 - val_loss: 0.8852 - val_accuracy: 0.6767
```



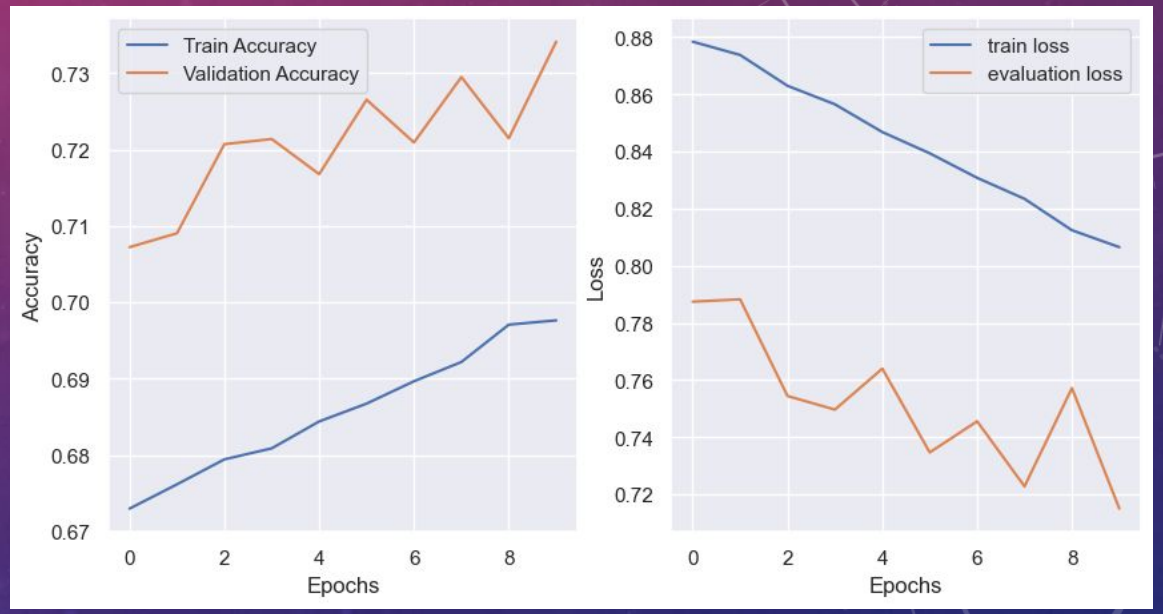
CUARTO ENTRENAMIENTO

```
Epoch 1/10
    2/13491 [..............................] - ETA: 24:40 - loss: 0.8795 - accuracy: 0.7500WARNING:tensorflow:Callbacks method `on_train_bat
13491/13491 [==============================] - 1668s 124ms/step - loss: 0.9869 - accuracy: 0.6360 - val_loss: 0.8653 - val_accuracy: 0.6849
Epoch 2/10
13491/13491 [==============================] - 1683s 125ms/step - loss: 0.9790 - accuracy: 0.6398 - val_loss: 0.8610 - val_accuracy: 0.6847
Epoch 3/10
13491/13491 [==============================] - 1690s 125ms/step - loss: 0.9642 - accuracy: 0.6429 - val_loss: 0.8616 - val_accuracy: 0.6857
Epoch 4/10
13491/13491 [==============================] - 1691s 125ms/step - loss: 0.9542 - accuracy: 0.6489 - val_loss: 0.8582 - val_accuracy: 0.6830
Epoch 5/10
13491/13491 [==============================] - 1692s 125ms/step - loss: 0.9482 - accuracy: 0.6488 - val_loss: 0.8346 - val_accuracy: 0.6955
Epoch 6/10
13491/13491 [==============================] - 1693s 126ms/step - loss: 0.9322 - accuracy: 0.6549 - val_loss: 0.8309 - val_accuracy: 0.6927
Epoch 7/10
13491/13491 [==============================] - 1696s 126ms/step - loss: 0.9195 - accuracy: 0.6588 - val_loss: 0.8081 - val_accuracy: 0.7043
Epoch 8/10
13491/13491 [==============================] - 1697s 126ms/step - loss: 0.9083 - accuracy: 0.6619 - val_loss: 0.8014 - val_accuracy: 0.7046
Epoch 9/10
13491/13491 [==============================] - 1698s 126ms/step - loss: 0.9028 - accuracy: 0.6651 - val_loss: 0.8041 - val_accuracy: 0.7019
Epoch 10/10
13491/13491 [==============================] - 1699s 126ms/step - loss: 0.8912 - accuracy: 0.6692 - val_loss: 0.7813 - val_accuracy: 0.7105
```



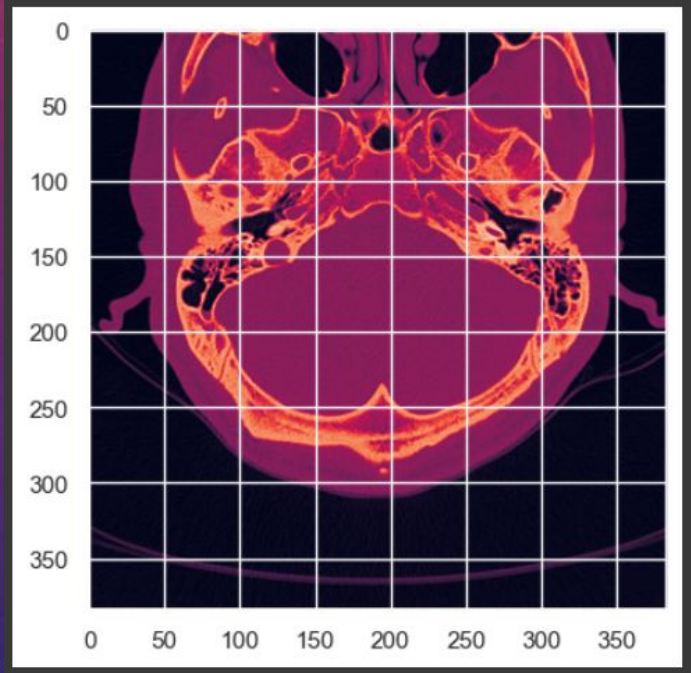
QUINTO ENTRENAMIENTO

```
Epoch 1/10
    2/13491 [..............................] - ETA: 23:55 - loss: 0.6412 - accuracy: 0.8125WARNING:tensorflow:Callbacks method `on_train_bat
13491/13491 [==============================] - 1679s 124ms/step - loss: 0.8785 - accuracy: 0.6730 - val_loss: 0.7875 - val_accuracy: 0.7072
Epoch 2/10
13491/13491 [==============================] - 1689s 125ms/step - loss: 0.8738 - accuracy: 0.6762 - val_loss: 0.7883 - val_accuracy: 0.7091
Epoch 3/10
13491/13491 [==============================] - 1690s 125ms/step - loss: 0.8630 - accuracy: 0.6794 - val_loss: 0.7544 - val_accuracy: 0.7207
Epoch 4/10
13491/13491 [==============================] - 1691s 125ms/step - loss: 0.8566 - accuracy: 0.6809 - val_loss: 0.7497 - val_accuracy: 0.7214
Epoch 5/10
13491/13491 [==============================] - 1679s 124ms/step - loss: 0.8468 - accuracy: 0.6844 - val_loss: 0.7640 - val_accuracy: 0.7168
Epoch 6/10
13491/13491 [==============================] - 1692s 125ms/step - loss: 0.8394 - accuracy: 0.6867 - val_loss: 0.7347 - val_accuracy: 0.7265
Epoch 7/10
13491/13491 [==============================] - 1709s 127ms/step - loss: 0.8308 - accuracy: 0.6897 - val_loss: 0.7456 - val_accuracy: 0.7210
Epoch 8/10
13491/13491 [==============================] - 1712s 127ms/step - loss: 0.8235 - accuracy: 0.6922 - val_loss: 0.7228 - val_accuracy: 0.7295
Epoch 9/10
13491/13491 [==============================] - 1708s 127ms/step - loss: 0.8125 - accuracy: 0.6971 - val_loss: 0.7572 - val_accuracy: 0.7215
Epoch 10/10
13491/13491 [==============================] - 1707s 127ms/step - loss: 0.8065 - accuracy: 0.6977 - val_loss: 0.7150 - val_accuracy: 0.7341
```



PARA EVALUAR

```
img_to_predict = load_img(PATH + 'imgs_por_vertebra_test/1.2.826.0.1.3680043.31077/C1/1.jpg', color_mode="grayscale")
img_to_predict = img_to_predict.resize((384, 384))
plt.imshow(img_to_predict)
img_to_predict = img_to_array(img_to_predict)
img_to_predict = img_to_predict.reshape( -1,384, 384,1)
```



RESULTADOS

```
1.2.826.0.1.3680043.31077
Vertebra:  C1 : ['C20', 0.5] fractura:  0
Vertebra:  C2 : ['C30', 0.9895833333333334] fractura:  0
Vertebra:  C3 : ['C40', 0.6770833333333334] fractura:  1
Vertebra:  C4 : ['C50', 0.4791666666666667] fractura:  1
Vertebra:  C5 : ['C61', 0.3229166666666667] fractura:  1
Vertebra:  C6 : ['C70', 0.9895833333333334] fractura:  1
Vertebra:  C7 : ['C70', 1.0] fractura:  0
```

```
1.2.826.0.1.3680043.31151
Vertebra:  C1 : ['C21', 0.5416666666666666] fractura:  0
Vertebra:  C2 : ['C21', 0.5416666666666666] fractura:  0
Vertebra:  C3 : ['C30', 0.8333333333333334] fractura:  0
Vertebra:  C4 : ['C30', 0.5416666666666666] fractura:  0
Vertebra:  C5 : ['C50', 0.625] fractura:  0
Vertebra:  C6 : ['C60', 0.7083333333333334] fractura:  0
Vertebra:  C7 : ['C70', 0.5769230769230769] fractura:  0
```

```
1.2.826.0.1.3680043.31088
Vertebra:  C1 : ['C11', 0.6571428571428571] fractura:  1
Vertebra:  C2 : ['C21', 0.6] fractura:  1
Vertebra:  C3 : ['C21', 0.4] fractura:  0
Vertebra:  C4 : ['C40', 0.9428571428571428] fractura:  0
Vertebra:  C5 : ['C50', 0.6] fractura:  0
Vertebra:  C6 : ['C60', 0.5142857142857142] fractura:  0
Vertebra:  C7 : ['C70', 0.6756756756756757] fractura:  0
```

```
1.2.826.0.1.3680043.31083
Vertebra:  C1 : ['C1', 0.7] fractura:  0
Vertebra:  C2 : ['C20', 1.0] fractura:  0
Vertebra:  C3 : ['C60', 0.7333333333333333] fractura:  0
Vertebra:  C4 : ['C60', 0.9666666666666667] fractura:  0
Vertebra:  C5 : ['C60', 0.4666666666666667] fractura:  0
Vertebra:  C6 : ['C70', 0.6666666666666666] fractura:  0
Vertebra:  C7 : ['C1', 0.6] fractura:  1
```

```
1.2.826.0.1.3680043.31114
Vertebra:  C1 : ['C10', 0.6181818181818182] fractura:  0
Vertebra:  C2 : ['C30', 0.2] fractura:  0
Vertebra:  C3 : ['C50', 0.5272727272727272] fractura:  0
Vertebra:  C4 : ['C40', 0.9818181818181818] fractura:  0
Vertebra:  C5 : ['C50', 0.5454545454545454] fractura:  0
Vertebra:  C6 : ['C60', 0.4909090909090909] fractura:  0
Vertebra:  C7 : ['C70', 0.5964912280701754] fractura:  0
```

```
1.2.826.0.1.3680043.31205
Vertebra:  C1 : ['C70', 0.7333333333333333] fractura:  1
Vertebra:  C2 : ['C60', 0.5111111111111111] fractura:  0
Vertebra:  C3 : ['C50', 0.8] fractura:  0
Vertebra:  C4 : ['C40', 0.7555555555555555] fractura:  0
Vertebra:  C5 : ['C30', 0.6] fractura:  0
Vertebra:  C6 : ['C30', 0.37777777777777777] fractura:  0
Vertebra:  C7 : ['C10', 0.4782608695652174] fractura:  1
```

```
1.2.826.0.1.3680043.31264
Vertebra:  C1 : ['C21', 0.4810126582278481] fractura:  0
Vertebra:  C2 : ['C21', 0.7468354430379747] fractura:  0
Vertebra:  C3 : ['C21', 0.3924050632911392] fractura:  0
Vertebra:  C4 : ['C30', 0.4936708860759494] fractura:  0
Vertebra:  C5 : ['C60', 0.7848101265822784] fractura:  0
Vertebra:  C6 : ['C70', 0.7848101265822784] fractura:  0
Vertebra:  C7 : ['C70', 1.0] fractura:  0
```

```
1.2.826.0.1.3680043.31245
Vertebra:  C1 : ['C21', 0.4166666666666667] fractura:  0
Vertebra:  C2 : ['C30', 0.5] fractura:  0
Vertebra:  C3 : ['C30', 0.5833333333333334] fractura:  0
Vertebra:  C4 : ['C40', 1.0] fractura:  0
Vertebra:  C5 : ['C50', 0.75] fractura:  0
Vertebra:  C6 : ['C60', 0.7777777777777778] fractura:  0
Vertebra:  C7 : ['C1', 0.358974358974359] fractura:  1
```

```
1.2.826.0.1.3680043.31328
Vertebra:  C1 : ['C10', 0.746031746031746] fractura:  0
Vertebra:  C2 : ['C20', 0.4444444444444444] fractura:  0
Vertebra:  C3 : ['C30', 0.6984126984126984] fractura:  0
Vertebra:  C4 : ['C40', 0.5396825396825397] fractura:  0
Vertebra:  C5 : ['C50', 0.3492063492063492] fractura:  0
Vertebra:  C6 : ['C60', 0.8412698412698413] fractura:  1
Vertebra:  C7 : ['C60', 0.4411764705882353] fractura:  0
```

```
1.2.826.0.1.3680043.31351
Vertebra:  C1 : ['C20', 0.4444444444444444] fractura:  0
Vertebra:  C2 : ['C20', 0.4166666666666667] fractura:  0
Vertebra:  C3 : ['C30', 0.7222222222222222] fractura:  0
Vertebra:  C4 : ['C50', 0.6388888888888888] fractura:  0
Vertebra:  C5 : ['C70', 0.3888888888888889] fractura:  1
Vertebra:  C6 : ['C70', 0.5] fractura:  1
Vertebra:  C7 : ['C1', 1.0] fractura:  0
```

EVALUANDO

```
#MODEL A5 Evaluados con datos de 100 pacientes distintos a los de entrenamiento
model.evaluate(val_ds)
```

```
2917/2917 [==============================] - 131s 45ms/step - loss: 2.3558 - accuracy: 0.4321
[2.355811595916748, 0.4320802390575409]
```

```
#MODEL A5 Evaluando los datos con lo que se probo entrenamiento a entrenamiento
model.evaluate(val_ds)
```

```
3373/3373 [==============================] - 153s 45ms/step - loss: 0.7150 - accuracy: 0.7341
[0.7150195837020874, 0.734146237373352]
```

CONSUMO DE RECURSOS EN LOS ENTRENAMIENTOS

- Incluso utilizando la librería de keras que permite cargar las imágenes utilizando el disco duro se sigue consumiendo casi la totalidad de memoria RAM en el proceso.
- Se entrenó al modelo alrededor de 24 horas.

PROBLEMAS

- La limitación en la capacidad de procesamiento afecta en gran medida el poder realizar un entrenamiento mas riguroso.
- La perdida de datos que se obtiene en la clasificación de las imágenes afecta en gran medida al entrenamiento, ya que se realiza a ojo la determinación de las imágenes clasificando por vertebras cada una.
- La falta de información del dataset afecta el poder realizar una mejor clasificación y evitar la perdida de datos.

CONCLUSIONES

- Se encontró overfiting.
- El modelo al contar con pocas imágenes clasificadas como fractura se le dificulto aprender.
- Aprendió muy bien a identificar vértebras sin fractura.
- Importancia de recursos fisicos.